

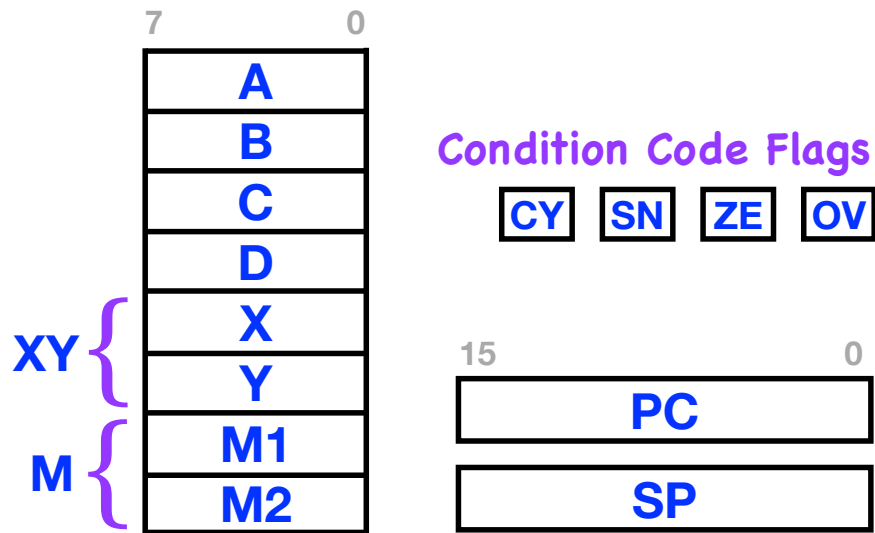
EveCore

Microcontroller Core

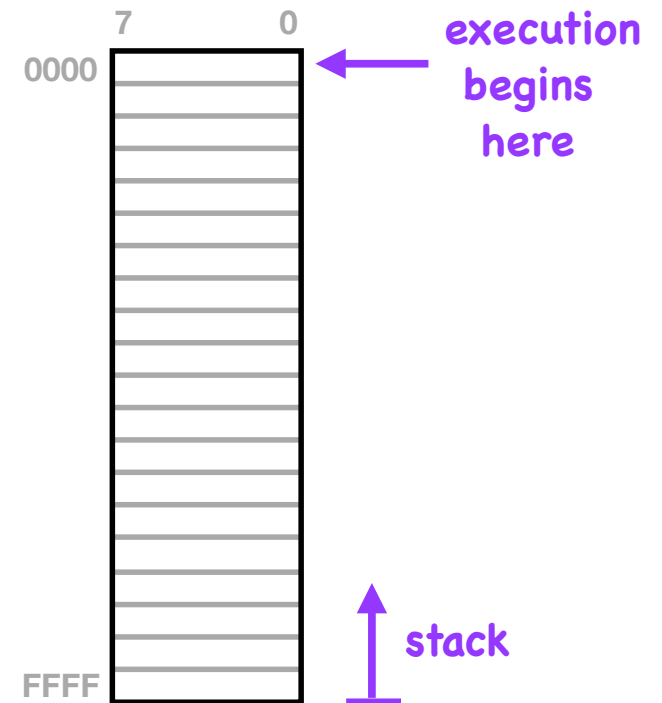
Architectural Overview

Blitz64.org/EveCore

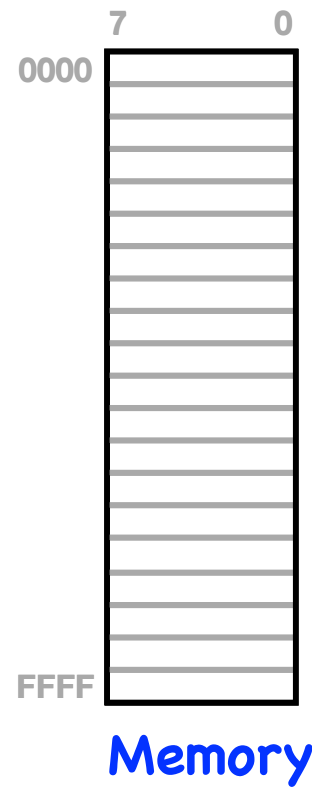
Registers



Memory



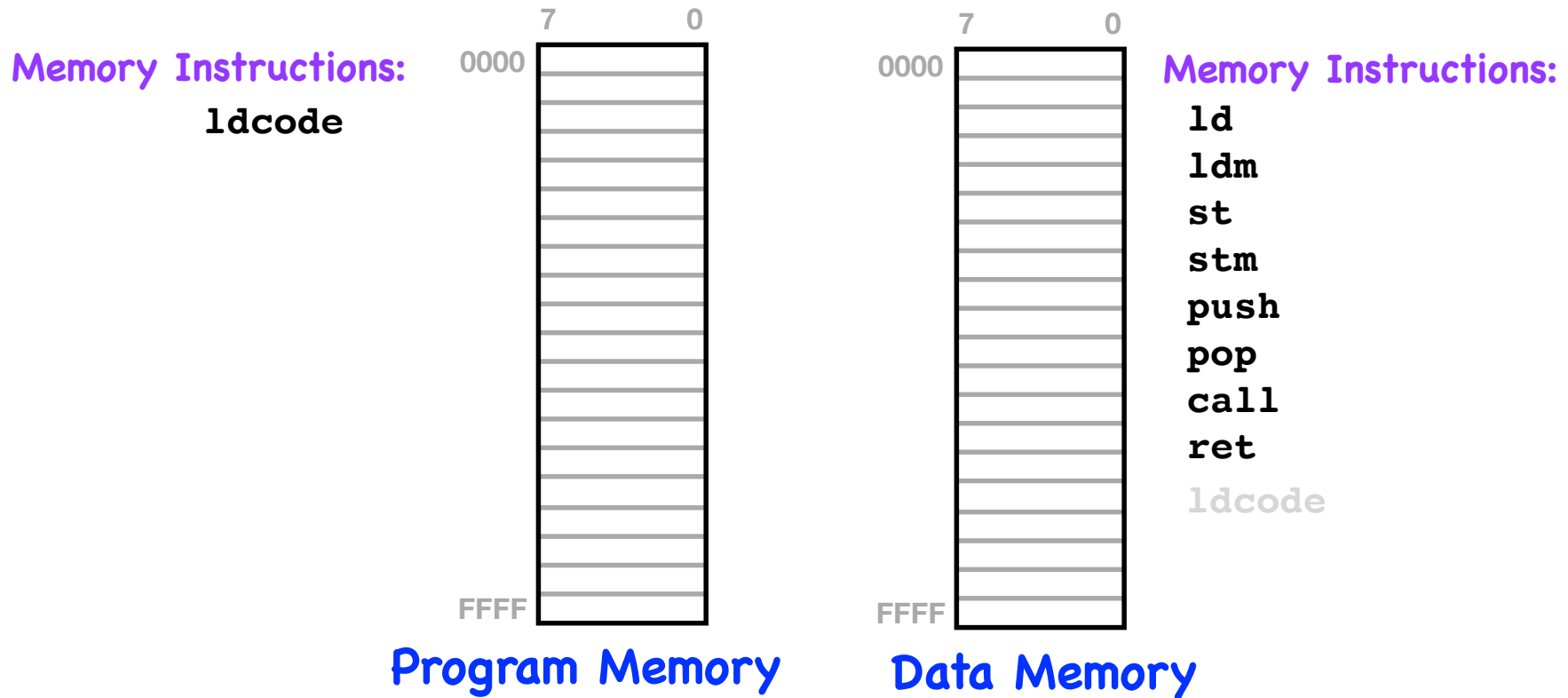
von Neumann Implementation



Memory Instructions:

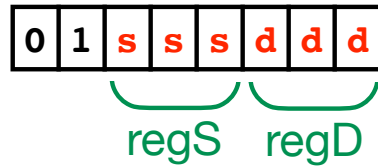
ld
ldm
st
stm
push
pop
call
ret
ldcode

Harvard Implementation

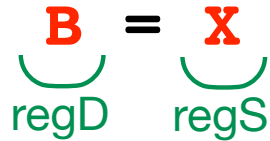


mov

Binary encoding



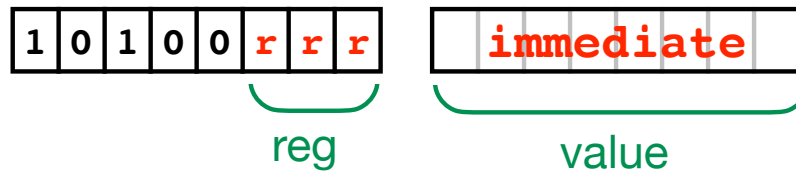
Assembly Example



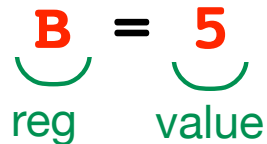
000 : A
001 : B
010 : C
011 : D
100 : X
101 : Y
110 : M1
111 : M2

movi

Binary encoding



Assembly Example



*Immediate
Value
Examples:*
123
0x4e
myLabel
myVal*4

ld

Binary encoding



Assembly Example

B = [**0x12cd**]

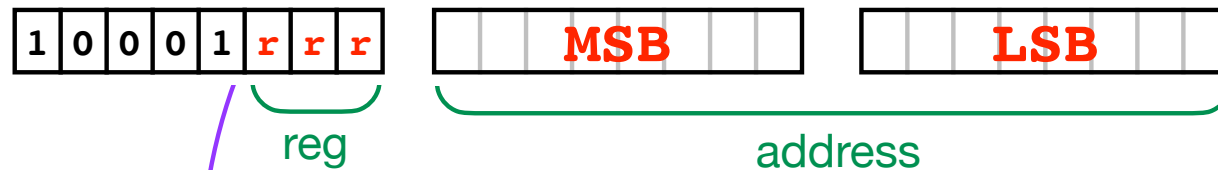
B is under a bracket labeled `reg`. **0x12cd** is under a bracket labeled `address`.

Address Examples:

12345
0x3a4e
Label_4
<expression>

st

Binary encoding



Assembly Example

[**myVar**] = **M1**

myVar is under a bracket labeled `address`. **M1** is under a bracket labeled `reg`.

ldm

Binary encoding



Assembly Example

A = [**M** + **myOffset**]
immediate

stm

Binary encoding



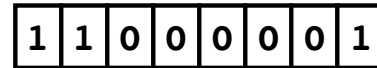
Assembly Example

[M + myArray] = A

immediate

ldcode

Binary encoding

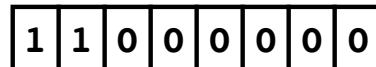


Assembly Example

A = code[XY]

ldsp

Binary encoding



MSB



LSB

address

Assembly Example

SP = 0xf00

address

and

Binary encoding

1	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Assembly Example

A = A & B

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

or

Binary encoding

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

Assembly Example

A = A | B

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

xor

Binary encoding

1	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Assembly Example

A = A ^ B

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

not

Binary encoding

1	0	1	1	0	0	1	1
---	---	---	---	---	---	---	---

Assembly Example

A = !A

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

add

Binary encoding

1	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

Assembly Example

A = A + B

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: modified

addc

Binary encoding

1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Assembly Example

A = A + B + CY

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: modified

sub

Binary encoding

1	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

Assembly Example

A = A - B

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: modified

subc

Binary encoding

1	0	1	1	0	1	1	1
---	---	---	---	---	---	---	---

Assembly Example

A = A - B - CY

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: modified

neg

Binary encoding

1	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Assembly Example

A = -A

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: modified

clr

Binary encoding

1	0	1	1	1	0	0	1
---	---	---	---	---	---	---	---

Assembly Example

A = 0

Condition Codes

CY: 0

SN: 0

ZE: 1

OV: 0

shl

Binary encoding

1	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

Assembly Example

A = A << 1

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: 0

shlc

Binary encoding

1	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---

Assembly Example

A = A <<< 1

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: 0

shr

Binary encoding

1	0	1	1	1	1	0	0
---	---	---	---	---	---	---	---

Assembly Example

A = A >> 1

Condition Codes

CY: modified

SN: 0

ZE: modified

OV: 0

shrc

Binary encoding

1	0	1	1	1	1	0	1
---	---	---	---	---	---	---	---

Assembly Example

A = A >>> 1

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: 0

incr

Binary encoding

1	0	1	1	1	1	1	0
---	---	---	---	---	---	---	---

Assembly Example

A = A + 1

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

decr

Binary encoding

1	0	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Assembly Example

A = A - 1

Condition Codes

CY: 0

SN: modified

ZE: modified

OV: 0

cmp

Binary encoding

1	1	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Assembly Example

cmp A:B

Condition Codes

CY: modified

SN: modified

ZE: modified

OV: modified

add16

Binary encoding

1	0	1	0	1	s	s	d
---	---	---	---	---	---	---	---

--	--	--	--	--	--	--	--

MSB

--	--	--	--	--	--	--	--

LSB

immediate

Assembly Example

XY = **XY** + **3**

regD regS immediate

ss	00:	XY
	01:	M
	10:	0
	11:	A
d	0:	XY
	1:	M

call

Binary encoding

1	1	0	0	0	0	1	1
---	---	---	---	---	---	---	---

--	--	--	--	--	--	--	--	--	--

MSB

--	--	--	--	--	--	--	--	--	--

LSB

address

Assembly Example

call **myFunction**

address

ret

Binary encoding

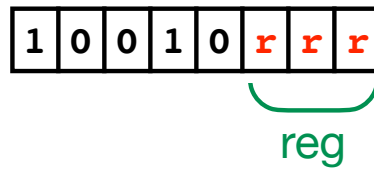
1	1	0	0	0	1	0	0
---	---	---	---	---	---	---	---

Assembly Example

ret

push

Binary encoding



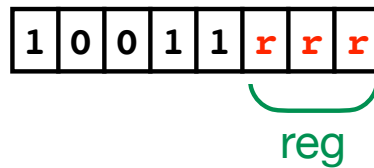
Assembly Example

push **B**

reg

pop

Binary encoding



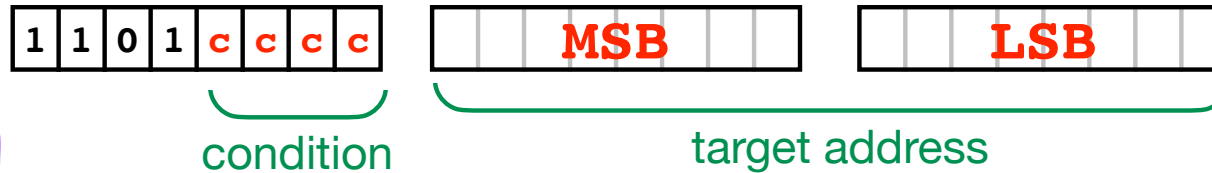
Assembly Example

pop **M1**

reg

branch

Binary encoding



Assembly Example

if **<=** **goto** **myLabel**

condition target address

condition

0000: ==, ZE	0100: >	1000: >u	1100: !OV
0001: !=, !ZE	0101: >=	1001: >=u, !CY	1101: OV
0010: <	0110: <u, CY	1010: !SN	1110: true
0011: <=	0111: <=u	1011: SN	1111: <undef>

gotoxy

Binary encoding

1	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

Assembly Example

goto XY

nop

Binary encoding

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Assembly Example

nop

in

Binary encoding

1	1	0	0	0	1	0	1
---	---	---	---	---	---	---	---

	I/O	Reg		
--	-----	-----	--	--

Register Number

Assembly Example

A = in 5

Register Number

out

Binary encoding

1	1	0	0	0	1	1	0
---	---	---	---	---	---	---	---

	I/O	Reg		
--	-----	-----	--	--

Register Number

Assembly Example

out 5 = A

Register Number

Assembler Directives

.flash		<i>Everything following will be placed in program memory</i>
.ram		<i>Everything following will be placed in data memory</i>
.byte	<i>Expr</i>	<i>Reserve and initialize 1 byte in memory</i>
.halfword	<i>Expr</i>	<i>Reserve and initialize 2 bytes</i>
.word	<i>Expr</i>	<i>Reserve and initialize 4 bytes</i>
.string	<i>DoubleQuotedString</i>	
.skip	<i>Expr</i>	<i>Skip over <expr> bytes</i>
.equ	<i>Expr</i>	<i>The label is equated (set to) the value of the expression</i>